

Roadmap: You Are Here

Time	Part	Status
1:00–2:15	Part 4: Going Distributed	✓
2:30–3:15	Part 5: Economics & Sustainability	← You are here
3:15–3:45	Part 6: Design Space Exploration	
3:45–4:15	Part 7: TinyML to Frontier	
4:15–4:45	Part 8: Advanced Topics	
4:45–5:00	Part 9: Wrap-Up & Capstone	

TUTORIAL | TUTORIAL

Economics & Sustainability

1

Key Question

**How much does it *really* cost to train and serve an LLM—
in dollars, kilowatt-hours, and tonnes of CO₂?**

“A 1024-GPU H100 cluster costs \$30M in hardware. Electricity is surprisingly small (~10% of total). The dominant cost lever is utilization—idle GPUs cost the same as busy ones.”

Wall 17: The Capital Wall (TCO)

The Equation

$$\text{TCO} = \underbrace{\text{CapEx}}_{\text{hw + facility}} + \underbrace{\text{OpEx}_{\text{energy}}}_{\text{electricity}} + \underbrace{\text{OpEx}_{\text{maint}}}_{\text{staff}}$$

Key insight: GPUs are only ~40% of total CapEx.
Networking, cooling, facility, and staff add 50–150%.

```
econ = mlsysim.  
    EconomicsModel()  
r = econ.solve(  
    fleet=mlsysim.Systems  
        .Clusters.Research_256,  
    duration_days=30,  
    infrastructure_multiplier  
        =2.5)  
print(f"TCO: ${r.tco_usd:,.0  
    f}")
```

- `infrastructure_multiplier = 2.0–2.5` for full datacenter TCO
- Amortized over 3–5 year depreciation schedule

The Infrastructure Multiplier

Component	Cost (%)	Multiplier
GPU Hardware	40%	1.0×
Network (IB)	20%	—
Power & Cooling	15%	—
Facility / Land	10%	—
Staff / Operations	15%	—
Total	100%	2.5×

Pitfall: Budgeting only for GPUs underestimates TCO by 2–3×

Wall 18: The Sustainability Wall

The Equation

$$\text{CO}_2 = \text{Energy} \times \text{PUE} \times \text{CI}$$

Three levers:

1. **Energy** — Better MFU, smaller model
2. **PUE** — Liquid cooling (1.3 → 1.06)
3. **Carbon intensity** — Choose your grid

```
sus = mlsysim.  
    SustainabilityModel()  
r = sus.solve(  
    fleet=mlsysim.Systems  
        .Clusters.Research_256,  
    datacenter=mlsysim.Infra  
        .Grids.Quebec,  
    duration_days=30, mfu  
        =0.45)  
print (f"{r.  
        carbon_footprint_kg  
            /1000:.1f} tonnes  
        CO2")
```

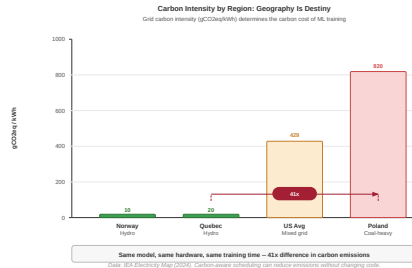
Carbon Intensity: The $41\times$ Gap

Region	Mix	gCO ₂
Quebec	Hydro	20
Sweden	Hydro+Nuc	25
US Avg	Mixed	390
Germany	Coal+Wind	380
Poland	Coal	820

Training the *same model*:

Poland: $\sim 800\text{ t}$ vs **Quebec:** $\sim 20\text{ t}$

$\Rightarrow 41\times$ difference.



Embodied Carbon: Manufacturing Dominates at the Edge

Cloud / Training

- Operational carbon dominates
- GPU runs 24/7 for months
- Embodied: $\sim 5\%$ of lifecycle

IoT / TinyML

- Manufacturing carbon dominates
- MCU uses microwatts
- Embodied: $>90\%$ of lifecycle
- Multiplied by millions of devices

Source: Gupta et al. (2022), “ACT: Designing Sustainable Computer Systems with an Architectural Carbon Modeling Tool”

Live Demo: TCO + Carbon Analysis

```
fleet = mlsysim.Systems.Clusters.Research_256
econ = mlsysim.EconomicsModel()
result = econ.solve(
    fleet=fleet, duration_days=30,
    datacenter=mlsysim.Infra.Grids.Quebec,
    mfu=0.45, infrastructure_multiplier=2.5,
    amortization_years=3.0)
print(f"TCO: ${result.tco_usd:,.0f}")
print(f"Carbon: {result.carbon_footprint_kg/1000:.1f} t")
```

Live Demo: The Carbon Geography Experiment

```
fleet = mlsysim.Systems.Clusters.Research_256
solver = mlsysim.SustainabilityModel()
for region in [mlsysim.Infra.Grids.Poland,
               mlsysim.Infra.Grids.Quebec]:
    r = solver.solve(fleet=fleet, duration_days=30,
                    datacenter=region, mfu=0.45)
    print(f"{r.region_name:>12}: "
          f"{r.carbon_footprint_kg/1000:.1f} t CO2")
```

Expected Output

```
Poland: 1,064 MWh, 872.5 tonnes CO2
Quebec: 1,064 MWh, 21.3 tonnes CO2
```

Same energy 41x less carbon Geography wins

Exercise 5a: Carbon-Aware Placement

Your Task (3 minutes)

Compare a 30-day training run on 512 H100s: **Poland** vs **Quebec**. How many tonnes of CO₂ saved?

```
fleet = mlsysim.Fleet(  
    name="Training Fleet",  
    node=mlsysim.Systems.Nodes.DGX_H100,  
    count=64,  
    fabric=mlsysim.Systems.Fabrics.InfiniBand_NDR)  
solver = mlsysim.SustainabilityModel()  
# YOUR CODE: solve for Poland and Quebec
```

Exercise 5b: Multi-Vendor TCO Shootout

Your Task (5 minutes)

Same workload: Llama-3 70B, 30-day training run, 512 accelerators.

Which cluster is cheapest for the **same throughput**?

```
econ = mlsysim.EconomicsModel()
for cluster_name in ["H100", "MI300X", "Gaudi3"]:
    hw = getattr(mlsysim.Hardware.Cloud, cluster_name)
    fleet = mlsysim.Fleet(name=cluster_name, hw=hw,
                          count=64)
    r = econ.solve(fleet=fleet, duration_days=30,
                  infrastructure_multiplier=2.5)
    print(f"{cluster_name:>8}: TCO=${r.tco_usd:,.0f}")
```

The cheapest accelerator depends on the workload's binding constraint.

Key Takeaway: Economics & Sustainability

1. **TCO $\neq$ GPU cost.** Infrastructure is 2–2.5 $\times$.
2. **Utilization is the cost lever.** Idle GPUs cost the same.
3. **Geography $>$ algorithms** for carbon reduction.

*“The cheapest watt is the one you don’t consume.
The cleanest watt is the one from a hydro dam.”*

TUTORIAL | TUTORIAL

Design Space Exploration

2

Key Question

**Given a budget and an SLA,
how do you find the *best* hardware configuration?**

The search space is combinatorial:

$|\text{hardware}| \times |\text{batch sizes}| \times |\text{precisions}| \times |\text{parallelism configs}|$
can easily exceed 10^4 configurations.

The DSE Pattern: Declare, Search, Rank

1. Declare the search space:

- ▶ Hardware: {A100, H100, H200, B200}
- ▶ Batch sizes: {1, 2, 4, ..., 64}
- ▶ Precisions: {fp16, int8, int4}

2. Search with an objective:

- ▶ minimize: tco_usd
- ▶ maximize: throughput

3. Rank subject to constraints:

- ▶ latency < 50 ms
- ▶ feasible == True

```
from mlsysim.core.dse import DSE

dse = DSE(
    space={
        "batch_size": [1, 4, 8, 16, 32],
        "precision": ["fp16", "int8"]
    },
    objective="maximize:
        throughput",
    constraints=["latency < 100"])
```

Analytical models $\Rightarrow$ each evaluation
takes < 1 ms $\Rightarrow$ exhaustive search.



1. 2. 3. 4. 5. 6. 7. 8. 9. 10. 11. 12. 13. 14. 15. 16. 17. 18. 19. 20. 21. 22. 23. 24. 25. 26. 27. 28. 29. 30. 31. 32. 33. 34. 35. 36. 37. 38. 39. 40. 41. 42. 43. 44. 45. 46. 47. 48. 49. 50. 51. 52. 53. 54. 55. 56. 57. 58. 59. 60. 61. 62. 63. 64. 65. 66. 67. 68. 69. 70. 71. 72. 73. 74. 75. 76. 77. 78. 79. 80. 81. 82. 83. 84. 85. 86. 87. 88. 89. 90. 91. 92. 93. 94. 95. 96. 97. 98. 99. 100. 101. 102. 103. 104. 105. 106. 107. 108. 109. 110. 111. 112. 113. 114. 115. 116. 117. 118. 119. 120. 121. 122. 123. 124. 125. 126. 127. 128. 129. 130. 131. 132. 133. 134. 135. 136. 137. 138. 139. 140. 141. 142. 143. 144. 145. 146. 147. 148. 149. 150. 151. 152. 153. 154. 155. 156. 157. 158. 159. 160. 161. 162. 163. 164. 165. 166. 167. 168. 169. 170. 171. 172. 173. 174. 175. 176. 177. 178. 179. 180. 181. 182. 183. 184. 185. 186. 187. 188. 189. 190. 191. 192. 193. 194. 195. 196. 197. 198. 199. 200. 201. 202. 203. 204. 205. 206. 207. 208. 209. 210. 211. 212. 213. 214. 215. 216. 217. 218. 219. 220. 221. 222. 223. 224. 225. 226. 227. 228. 229. 230. 231. 232. 233. 234. 235. 236. 237. 238. 239. 240. 241. 242. 243. 244. 245. 246. 247. 248. 249. 250. 251. 252. 253. 254. 255. 256. 257. 258. 259. 260. 261. 262. 263. 264. 265. 266. 267. 268. 269. 270. 271. 272. 273. 274. 275. 276. 277. 278. 279. 280. 281. 282. 283. 284. 285. 286. 287. 288. 289. 290. 291. 292. 293. 294. 295. 296. 297. 298. 299. 300. 301. 302. 303. 304. 305. 306. 307. 308. 309. 310. 311. 312. 313. 314. 315. 316. 317. 318. 319. 320. 321. 322. 323. 324. 325. 326. 327. 328. 329. 330. 331. 332. 333. 334. 335. 336. 337. 338. 339. 340. 341. 342. 343. 344. 345. 346. 347. 348. 349. 350. 351. 352. 353. 354. 355. 356. 357. 358. 359. 360. 361. 362. 363. 364. 365. 366. 367. 368. 369. 370. 371. 372. 373. 374. 375. 376. 377. 378. 379. 380. 381. 382. 383. 384. 385. 386. 387. 388. 389. 390. 391. 392. 393. 394. 395. 396. 397. 398. 399. 400. 401. 402. 403. 404. 405. 406. 407. 408. 409. 410. 411. 412. 413. 414. 415. 416. 417. 418. 419. 420. 421. 422. 423. 424. 425. 426. 427. 428. 429. 430. 431. 432. 433. 434. 435. 436. 437. 438. 439. 440. 441. 442. 443. 444. 445. 446. 447. 448. 449. 450. 451. 452. 453. 454. 455. 456. 457. 458. 459. 460. 461. 462. 463. 464. 465. 466. 467. 468. 469. 470. 471. 472. 473. 474. 475. 476. 477. 478. 479. 480. 481. 482. 483. 484. 485. 486. 487. 488. 489. 490. 491. 492. 493. 494. 495. 496. 497. 498. 499. 500. 501. 502. 503. 504. 505. 506. 507. 508. 509. 510. 511. 512. 513. 514. 515. 516. 517. 518. 519. 520. 521. 522. 523. 524. 525. 526. 527. 528. 529. 530. 531. 532. 533. 534. 535. 536. 537. 538. 539. 540. 541. 542. 543. 544. 545. 546. 547. 548. 549. 550. 551. 552. 553. 554. 555. 556. 557. 558. 559. 560. 561. 562. 563. 564. 565. 566. 567. 568. 569. 570. 571. 572. 573. 574. 575. 576. 577. 578. 579. 580. 581. 582. 583. 584. 585. 586. 587. 588. 589. 590. 591. 592. 593. 594. 595. 596. 597. 598. 599. 600. 601. 602. 603. 604. 605. 606. 607. 608. 609. 610. 611. 612. 613. 614. 615. 616. 617. 618. 619. 620. 621. 622. 623. 624. 625. 626. 627. 628. 629. 630. 631. 632. 633. 634. 635. 636. 637. 638. 639. 640. 641. 642. 643. 644. 645. 646. 647. 648. 649. 650. 651. 652. 653. 654. 655. 656. 657. 658. 659. 660. 661. 662. 663. 664. 665. 666. 667. 668. 669. 670. 671. 672. 673. 674. 675. 676. 677. 678. 679. 680. 681. 682. 683. 684. 685. 686. 687. 688. 689. 690. 691. 692. 693. 694. 695. 696. 697. 698. 699. 700. 701. 702. 703. 704. 705. 706. 707. 708. 709. 710. 711. 712. 713. 714. 715. 716. 717. 718. 719. 720. 721. 722. 723. 724. 725. 726. 727. 728. 729. 730. 731. 732. 733. 734. 735. 736. 737. 738. 739. 740. 741. 742. 743. 744. 745. 746. 747. 748. 749. 750. 751. 752. 753. 754. 755. 756. 757. 758. 759. 760. 761. 762. 763. 764. 765. 766. 767. 768. 769. 770. 771. 772. 773. 774. 775. 776. 777. 778. 779. 780. 781. 782. 783. 784. 785. 786. 787. 788. 789. 790. 791. 792. 793. 794. 795. 796. 797. 798. 799. 800. 801. 802. 803. 804. 805. 806. 807. 808. 809. 810. 811. 812. 813. 814. 815. 816. 817. 818. 819. 820. 821. 822. 823. 824. 825. 826. 827. 828. 829. 830. 831. 832. 833. 834. 835. 836. 837. 838. 839. 840. 84

Live Demo: Design Space Sweep

```
hw_list = [mlsysim.Hardware.Cloud.H100,
            mlsysim.Hardware.Cloud.MI300X,
            mlsysim.Hardware.Cloud.Gaudi3,
            mlsysim.Hardware.Cloud.B200]
results = mlsysim.Engine.sweep(
    llama, hw_list, batch_sizes=[1, 4, 16, 64],
    precisions=["fp16", "int8"], efficiency=0.5)
for r in sorted(results,
                 key=lambda x: x['profile'].latency.magnitude):
    p = r['profile']
    print (f"{r['hardware']:>8} bs={r['batch_size']:<3} "
          f"| {p.bottleneck:<8} | MFU={p.mfu:.3f}")
```

Live Demo: DSE with Objective & Constraints

```
from mlsysim.core.dse import DSE

dse = DSE(
    space={"batch_size": [1, 4, 8, 16, 32, 64],
          "precision": ["fp16", "int8"]},
    objective="maximize: throughput",
    constraints=["latency < 100"])

def evaluate(params):
    p = mlsysim.Engine.solve(llama, hw,
                             batch_size=params["batch_size"],
                             precision=params["precision"])
    return {"throughput": p.throughput.magnitude,
            "latency": p.latency.to("ms").magnitude}

best = dse.search(evaluate)
```

Live Demo: Batching Optimizer (Pareto Front)

```
opt = mlsysim.BatchingOptimizer()
result = opt.solve(
    model=llama, hardware=hw, seq_len=128,
    arrival_rate_qps=10.0,
    sla_latency_ms=20000.0, precision="fp16")
print(f"Optimal bs: {result.best_batch_size}")
for pt in result.pareto_front:
    print(f"  bs={pt['batch_size']:<4} "
          f"lat={pt['p99_latency'].m_as('ms'):.0f} ms")
```

Exercise: Budget-Constrained Design

Your Task (5 minutes)

Given a **\$1M budget**, find the best hardware + batch size + precision configuration for serving Llama-3-8B inference under a 50 ms latency SLA.

```
# Hint: Use Engine.sweep across hardware tiers,  
# filter by feasibility and latency SLA,  
# then pick the highest-throughput config  
# within budget (check unit_cost in Hardware)
```

Key Takeaway: Design Space Exploration

1. **Analytical models** make exhaustive DSE feasible (< 1 s for 10^4 configs)
2. **Pareto fronts** reveal the actual tradeoff structure
3. **Constraints first**: filter infeasible, then optimize

*“You cannot optimize what you cannot model.
You cannot model what you cannot measure.”*

TUTORIAL | TUTORIAL

TinyML to Frontier

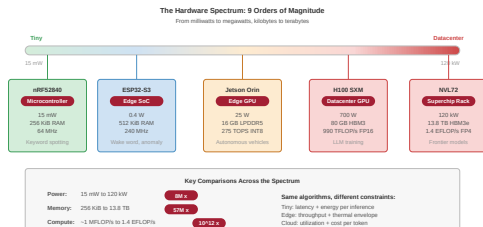
3

Key Question

**Can the same analytical framework model
a \$2 microcontroller *and* a \$3M GPU rack?**

The answer is yes—because the **Roofline model** is universal.
Only the numbers change. The physics is the same.

The 9-Order-of-Magnitude Scale Span



ML systems span 9 orders of magnitude in compute. The physics changes; the principles do not.

| Device | FLOPS | TDP |
|----------|-------|--------|
| nRF52840 | 64 M | 15 mW |
| ESP32-S3 | 500 M | 400 mW |
| H100 SXM | 989 T | 700 W |

Compute $\sim 10^7 \times$
Mem BW $\sim 10^7 \times$
Power $\sim 10^{4.7} \times$

Same Roofline equation. Nine orders of magnitude apart.

Flash vs SRAM: The TinyML Memory Wall

Cloud GPU (H100)

- Weights in HBM (80 GB)
- Activations in SRAM (50 MB L2)
- Bandwidth: 3.35 TB/s
- Model ≤ 80 GB $\Rightarrow$ fits

TinyML MCU (ESP32-S3)

- Weights in **Flash** (8 MB)
- Activations in **SRAM** (512 KiB)
- Flash BW: 80 MB/s
- Model ≤ 8 MB $\Rightarrow$ fits
- Model ≤ 512 KiB $\Rightarrow$ SRAM-only (12 $\times$ faster)

Key insight: mlsysim automatically selects the right memory tier:

1. Model fits in SRAM $\Rightarrow$ use SRAM bandwidth
2. Model in Flash $\Rightarrow$ use Flash bandwidth (bottleneck!)

Energy per Inference: μJ to Joules

| Device | TDP | Energy/Inf | Use Case |
|----------------|--------|-----------------------|------------------|
| nRF52840 | 15 mW | $\sim 50 \mu\text{J}$ | Keyword spotting |
| ESP32-S3 | 400 mW | $\sim 1 \text{ mJ}$ | Person detection |
| Jetson Orin NX | 25 W | $\sim 25 \text{ mJ}$ | Object detection |
| H100 SXM | 700 W | $\sim 50 \text{ J}$ | LLM inference |

- 6 orders of magnitude in energy per inference
- TinyML: battery life is the primary constraint
- Duty cycling (sleep 99% of the time) extends battery to years

Live Demo: nRF52840 vs ESP32 vs H100

```
tiny_model = mlsysim.Models.Tiny.KeywordSpotting
devices = [mlsysim.Hardware.Tiny.nRF52840,
            mlsysim.Hardware.Tiny.ESP32_S3,
            mlsysim.Hardware.Cloud.H100]
for d in devices:
    p = mlsysim.Engine.solve(tiny_model, d,
                             precision="int8", efficiency=0.3)
    print(f"{d.name:>15}: {p.latency.to('ms'):.2f~P}"
          f" | {p.bottleneck:<8}")
```

Observation

nRF52840 is memory-bound (Flash bottleneck). H100 finishes in μs —but wastes 700 W doing it. **Right-sizing hardware matters.** mlsysim also supports **Coral Edge TPU**,

Same Roofline, Different Physics

$$\text{Latency} = \max\left(\frac{\text{FLOPs}}{\text{Peak} \times \eta}, \frac{\text{Weights}}{\text{BW}}\right)$$

| Term | Cloud | TinyML |
|----------|-----------|---------|
| Peak | 989 TF | 500 MF |
| BW | 3.35 TB/s | 80 MB/s |
| Overhead | 0.01 ms | 1.0 ms |

```
# Same API, different hardware
for d in [mlsysim.Hardware.Cloud
        .H100,
        mlsysim.Hardware.Tiny.
        ESP32_S3]:
    p = mlsysim.Engine.solve(
        mlsysim.Models.Tiny
        .KeywordSpotting,
        d, precision="int8")
    print(f"{d.name}: {p.
            bottleneck}")
```

Both are memory-bound at batch size 1. Same equation, same diagnosis.

Exercise: Right-Sizing for IoT

Your Task (5 minutes)

A smart doorbell needs person detection at <100 ms latency on a coin-cell battery (500 mAh @ 3V = 1.5 Wh). Compare ESP32-S3 vs nRF52840: which device can run inference for a full year at 1 inference/minute?

```
import mlsysim

model = mlsysim.Models.Tiny.PersonDetection
for hw in [mlsysim.Hardware.Tiny.ESP32_S3,
           mlsysim.Hardware.Tiny.nRF52840]:
    p = mlsysim.Engine.solve(model, hw,
                             precision="int8", efficiency=0.3)
    energy_j = p.energy.to("J").magnitude
    inferences_per_year = 60 * 24 * 365
```

Key Takeaway: TinyML to Frontier

1. The Roofline model spans **9 orders of magnitude**
2. **Right-size** hardware to the workload, not the other way around
3. At TinyML scale, **energy and memory** dominate; at cloud scale, **cost and communication**

“The best accelerator is the smallest one that meets your latency and accuracy SLA.”

TUTORIAL | TUTORIAL

Advanced Topics

4

Key Question

**How do you compose multiple analytical models
into an end-to-end system analysis?**

Single-wall analysis answers “what is the bottleneck?”

Pipeline composition answers “what is the total cost of the whole stack?”

The Pipeline Composition Pattern

```
from mlsysim.core.pipeline import Pipeline
from mlsysim.core.solver import (
    ScalingModel, DistributedModel, EconomicsModel)

# Chain: Algorithm -> Fleet -> Economics
pipe = Pipeline([
    ScalingModel(),          # Wall 11: compute budget
    DistributedModel(),      # Wall 14: comms overhead
    EconomicsModel(),        # Wall 17: total cost
])
print(pipe.explain())      # shows DAG + gaps
```

`explain()` shows:

Running the Pipeline

```
from mlsysim.core.constants import Q_  
  
results = pipe.run(  
    compute_budget=Q_("1e21 FLOP"),  
    fleet=mlsysim.Systems.Clusters.Research_256,  
    duration_days=30,  
    datacenter=mlsysim.Infra.Grids.Quebec,  
    mfu=0.45,  
    infrastructure_multiplier=2.5)  
  
for stage_name, result in results.items():  
    print(f"\n--- {stage_name} ---")  
    print(result)
```

Wall 21: Sensitivity Analysis

```
solver = mlsysim.SensitivitySolver()
result = solver.solve(
    model=llama, hardware=hw, precision="fp16",
    perturbation_pct=10.0, efficiency=0.5)
print(f"Binding: {result.binding_constraint}")
for p, s in result.sensitivities.items():
    tag = "<<<" if p == result.binding_constraint else ""
    print(f"    {p:>20}: {s:+.4f}    {tag}")
```

Rule: Invest in the parameter with the *largest* $|\partial T / \partial p|$.

Improving a non-binding parameter yields **zero** measurable gain.

Wall 22: Inverse Roofline (SynthesisSolver)

```
from mlsysim.core.constants import Q_  
  
solver = mlsysim.SynthesisSolver()  
result = solver.solve(  
    model=llama, target_latency=Q_("50 ms"),  
    precision="fp16", efficiency=0.5)  
print(f"Required BW:      {result.required_bw:.0f~P}")  
print(f"Required FLOPS: {result.required_flops:.1f~P}")
```

Usage: “I need 50 ms TTFT. What hardware do I need?”

⇒ Derive specs from SLA, then match to real accelerators.

Fallacies & Pitfalls (Patterson Tradition)

Fallacy: “Cheaper hardware is always more cost-effective”

Reality: A $2\times$ cheaper GPU that takes $3\times$ longer has *higher* TCO.

$\text{TCO} = \text{CapEx} + \text{OpEx}$; slower hardware burns more electricity.

Fallacy: “More FLOPS = proportionally faster”

Reality: A100 $\rightarrow$ H100 is $6.3\times$ peak FLOPS but only $\sim 2\times$ for memory-bound LLM inference. The binding constraint is bandwidth, not compute.

Pitfall: Optimizing a non-binding parameter

If inference is memory-bound, doubling FLOPS gives 0% speedup.

Use `SensitivitySolver` to find the binding constraint *first*.

Pitfall: Ignoring the infrastructure multiplier

GPU cost is 40% of total datacenter TCO. Budgeting for GPUs alone

The Iron Law Revisited: All Five Terms

$$T = \frac{\text{FLOPs}}{N \times \text{Peak} \times \text{MFU} \times \eta_{\text{scale}} \times \text{Goodput}}$$

| Term | What reduces it | Wall |
|-----------------------|----------------------------------|---------|
| N | Budget (buy more GPUs) | Wall 17 |
| Peak | GPU generation (H100→B200) | Wall 1 |
| MFU | FlashAttention, kernel fusion | Wall 3 |
| η_{scale} | Network BW, gradient compression | Wall 14 |

Exercise: Binding Constraint Analysis

Your Task (5 minutes)

Use `SensitivitySolver` on Llama-3-8B + H100. Which parameter is binding?
Now switch to `batch_size=64` and re-run. Does the binding constraint change?

```
solver = mlsysim.SensitivitySolver()
# Batch size 1
r1 = solver.solve(llama, hw, efficiency=0.5)
print(f"bs=1: binding = {r1.binding_constraint}")
# Batch size 64
p64 = mlsysim.Engine.solve(llama, hw, batch_size=64)
print(f"bs=64: bottleneck = {p64.bottleneck}")
```

TUTORIAL | TUTORIAL

Wrap-up & Future

5

The 22 Walls of ML Systems

| # | Wall | # | Wall |
|----|----------------|----|-------------------------|
| 1 | Compute | 12 | Reasoning (Emerging) |
| 2 | Memory | 13 | Fidelity (Compression) |
| 3 | Software (MFU) | 14 | Communication |
| 4 | Serving | 15 | Fragility (Reliability) |
| 5 | Batching (KV) | 16 | Multi-tenant (Queueing) |
| 6 | Streaming | 17 | Capital (TCO) |
| 7 | Tail Latency | 18 | Sustainability |
| 8 | Ingestion | 19 | Checkpoint |
| 9 | Transformation | 20 | Safety (Privacy) |
| 10 | Locality | 21 | Sensitivity |
| 11 | Complexity | 22 | Synthesis (Inverse) |

What `mlsysim` Does *Not* Model

Not modeled (v0.1.0):

- Cache effects / tiling / fusion
- Real network congestion / jitter
- CUDA kernel scheduling
- Multi-model co-location
- Compiler optimizations

By design:

- Not a cycle-accurate simulator
- Not a profiler replacement
- Not a deployment tool
- **Is:** a first-principles analytical reasoning tool

Use it for:

- “Which constraint is binding?”
- “Is this hardware sufficient?”
- “What should I try first?”

v0.2.0 Roadmap

Models & Hardware

1. **MoE support** — Expert routing + parallelism
2. **GQA / MQA** — Grouped-query KV cache
3. **Network congestion** — Contention-aware collectives

Tooling & Integration

4. **Spot pricing** — Cloud cost optimization
5. **Browser exercises** — Live exploration
6. **HuggingFace Hub** — Auto-import models

Contributions welcome: `github.com/harvard-edge/mlsysim`

Design Challenge: Capstone Exercise

The Problem

\$5M budget. Serve Llama-3 70B at **1,000 QPS** with **<100 ms TTFT** in **two regions** (US-East + EU-West). Design the fleet.

You must specify:

1. Hardware choice (which GPU? how many?)
2. Parallelism strategy (TP $\times$ PP)
3. Precision (FP16? INT8? FP8?)
4. Geographic placement (carbon?)
5. Redundancy (replicas per region?)

```
import mlsysim

# Capstone starter code
model = mlsysim.Models.Language.
    Llama3_70B
hw = mlsysim.Hardware.H100

# Step 1: Can it fit?
p = mlsysim.Engine.solve(
    model, hw, precision="fp16")
```

Your Turn: Name That Wall

Think of one ML system in your own work.

Write down:

1. What is the system? (model + hardware + use case)
2. Which of the 22 walls is the **binding constraint**?
3. What would you change to move the wall?

2 minutes. Then we will hear from 3 volunteers.

Resources & Next Steps

Get Started

- `pip install mlsysim`
- GitHub: `harvard-edge/mlsysim`
- Website tutorials, examples, and API reference
- Full docs: `mlsysim.readthedocs.io`

The Textbook

- *Machine Learning Systems*
- Volume I: Foundations (single node)
- Volume II: Systems at Scale (fleet)

Key Papers

- Williams et al. (2009)
Roofline Model
- Chowdhery et al. (2022)
PaLM / MFU
- Hoffmann et al. (2022)
Chinchilla Scaling
- Patterson et al. (2021)
Carbon & Training
- Kwon et al. (2023)
PagedAttention